

The Echo Network

A Constructal, Post-Quantum Infrastructure Economy Built on Phinary Accounting

Eric James Wolverton
theechos.net

Abstract

The Echo Network is a decentralized infrastructure economy in which money, file transport, storage, computation, and resilience are expressed through a common envelope architecture. This paper redrafts the system with the constructal law in mind: persistent flow systems tend to evolve architectures that provide easier access to the currents that move through them. In Echo, the currents are value, files, proofs, storage challenges, computation tasks, and recovery traffic. The architectural response is a phi-fractal mesh of availability cells, phinary receipts, immutable envelopes, and a Phi-STARK commitment tree. Value is represented by exact phinary arithmetic, state transitions are carried by envelopes, and accepted actions generate debit, credit, and EchoProof records. The result is a post-quantum, proof-gated, self-accounting infrastructure network.

The paper also includes a simulation-guided design iteration. Five global symbolic simulations were run under increasing churn, attack pressure, payment load, and file-transfer load. At each stage the constructal criterion was applied: where flow resistance, congestion, or pending work appeared, the architecture was adjusted to increase access by widening phi neighbor degree, expanding availability-cell mirrors, and improving branch routes. Across the five iterations, the simulated network preserved safety invariants: no accepted double spends, no replay acceptances, no ledger math errors, and no storage overflow. The simulation is not presented as a production proof; it is a design instrument used to refine the Phase One architecture.

Finally, Phi-Life is retained as a research extension rather than a core safety claim. The envelope substrate may be interpreted as a generalized phi-fractal cellular automaton in which Echoes act as economic energy, storage as memory, computation as metabolism, routing as communication, and envelope fracture as reproduction. This paper does not claim that intelligence or technological singularity is guaranteed. It states the narrower, testable proposition: the architecture can host autonomous economic agents that persist, transact, replicate, and adapt under proof-gated resource constraints.

1. Plain-English Overview

The Echo Network is a network that pays useful participants. If a device helps move information, it can be paid. If it stores information, it can be paid. If it performs computation, it can be paid. If it helps the network recover from outage, attack, or degraded routing, it can be paid. The network is therefore not only a payment system. It is an economy for maintaining digital infrastructure.

The simplest way to understand Echo is this: every useful action becomes an envelope. A payment is an envelope. A file transfer is an envelope. A storage proof is an envelope. A computation result is an envelope. The envelope moves through the mesh, proves what happened, settles if valid, and leaves behind an accounting proof.

The network uses phinary value, based on the golden ratio. The attached Phi mathematics paper emphasizes a modest but important principle: recursive invariants can reveal large structure from small rules; Phi is useful not because it abolishes ordinary mathematics, but because its identity $\Phi^2 = \Phi + 1$ generates powerful recursive order. Echo applies that philosophy to accounting and routing: small exact rules organize large distributed behavior.

Constructal law adds a complementary principle. A system that persists must evolve to provide easier access to the currents that flow through it. Echo's currents are value, files, proofs, tasks, and recovery traffic. When the network is attacked or congested, the system should not merely survive passively. It should create incentives and routes that make future flow easier.

2. Contributions

This paper makes six contributions.

First, it presents Echo as a constructal infrastructure economy: money, file movement, storage, computation, and resilience are treated as flows through a finite network that must improve access over time.

Second, it defines a Phase One architecture consisting of a Phi-STARK tree, phinary payments, and file-transfer envelopes.

Third, it states safety invariants for exact value conservation, nullifier uniqueness, replay rejection, and proof-gated settlement.

Fourth, it gives a five-iteration global simulation under increasing churn and adversarial pressure, using the constructal criterion to adjust routing degree, cell size, mirror quorum, and recovery behavior.

Fifth, it incorporates the Phi mathematics manuscript as the conceptual basis for invariant recognition, recursive closure, and phinary representation.

Sixth, it retains Phi-Life as a research extension: a possible autonomous-agent ecology emerging from the same envelope substrate, while avoiding claims that intelligence or singularity is guaranteed.

3. Scientific Background

Constructal theory, associated with Adrian Bejan, studies how flow architectures emerge and change to provide easier access to currents under finite-size constraints. In this paper, the concept is used as an engineering design heuristic: if a mesh is congested, partitioned, or inefficient, widen or reshape access paths rather than merely retrying the same path. The claim here is not that Echo proves constructal law. The claim is that constructal thinking provides a useful design criterion for Echo's routing, storage, and recovery layers.

Cellular automata, including Conway's Game of Life, show that simple local rules can generate complex global patterns. Artificial-life research studies how digital systems may support persistence, reproduction, adaptation, and open-ended evolution. Echo uses this literature conservatively. It does not assert that intelligence exists merely because the network is self-similar. It provides an envelope-based environment in which autonomous economic agents could be built, tested, and measured.

Post-quantum cryptography provides the security direction. Production Echo uses quantum-resistant signatures and key establishment, such as NIST-standard ML-DSA, SLH-DSA, and ML-KEM where appropriate. STARK-style transparent proofs are used as a conceptual model for scalable computational integrity and append-only proof commitments.

4. Core Objects

The network is built from five core objects.

A Receipt is resting value. It has an owner, a phinary value, a parent reference, a state, and a deterministic nullifier commitment.

An Envelope is value or information in motion. It contains parent references, destination, payload commitment, expiry, proof commitments, nullifier, and settlement metadata.

An EchoProof is accounting memory. It binds the consumed debit, the created credit, the nullifier, the envelope identifier, and the verification result.

A Phi Availability Cell is a mirrored logical destination. It consists of phi-spaced nodes selected deterministically from the network. A cell is live when an honest quorum is available.

A Phi-STARK Tree is the append-only historical commitment structure. In Phase One it may be implemented as a deterministic phi-branching hash tree with STARK interfaces reserved. Later versions replace or augment hash-only commitments with transparent computational-integrity proofs.

5. Exact Phinary Value

Echo represents value with powers and canonical sums of powers of Phi. The fundamental identity is: Φ^k

$$= \Phi^{(k-1)} + \Phi^{(k-2)}.$$

A receipt fracture is valid only if value is conserved exactly. Consensus must not depend on floating point. A production implementation therefore represents values as integer pairs $A + B \Phi$ inside $Z[\Phi]$, together with canonical phinary exponent sets. Residual value below the protocol precision bound is not discarded; it is recorded as explicit silt.

The attached Phi mathematics manuscript is important here because it states the proper stance: phinary arithmetic is interesting because recursive invariants provide explanatory compression, not because they repeal complexity theory or thermodynamics. Echo follows the same stance. It uses Phi to organize exact accounting and self-similar routing, not to make impossible claims.

6. Phi Envelopes

Every state transition is represented as an immutable envelope. A payment envelope carries target value and change commitments. A file-transfer envelope carries file manifest, chunk commitments, and delivery proof metadata. A storage envelope carries shard commitments and challenge proofs. A compute envelope carries task commitments, result commitments, and verification evidence.

An envelope may progress through exactly one forward path:

AVAILABLE -> PLEDGED -> SETTLED

AVAILABLE -> PLEDGED -> REFUNDED

AVAILABLE -> INVALID

No state transition moves backward. This monotonicity is the core defense against settlement races. If a destination cannot safely settle before expiry, refund becomes legal after expiry. If a proof is malformed, the envelope is invalid.

7. Phi Availability Cells and Mirrors

A decentralized network cannot promise that a single offline device will receive a live packet. Echo therefore routes to logical cells. A wallet, file, shard, computation task, proof log, or destination address resolves to a Phi Availability Cell.

The cell is selected by deterministic hashing and phi-spaced offsets. The selected mirrors are not a central server. They are a temporary local representation of a logical object. A quorum of mirrors may certify a phase of an envelope.

Constructal adjustment enters here. If the cell is too small under churn, the system expands the cell. If routes are too narrow under attack, the system adds longer phi/Fibonacci shells. If a mirror class becomes congested, reward rates favor underused access paths. The design objective is not arbitrary growth. It is easier access for the actual currents moving through the network.

8. The Phi-STARK Tree

The Phi-STARK Tree is the commitment layer for historical state. Phase One implements it first as a phi-branching hash tree. Each accepted envelope creates a leaf:

$$\text{leaf} = H(\text{envelope_id}, \text{nullifier}, \text{debit_commitment}, \text{credit_commitment}, \text{proof_hash}).$$

Leaves are grouped using Fibonacci branch widths such as 1, 2, 3, 5, 8, and 13. Each branch root commits to ordered child roots. Each epoch produces one public root. The tree provides inclusion proofs, append-only auditability, and compact synchronization between peers.

The word STARK is used with care. Phase One does not need to implement a full zero-knowledge proving system. It must reserve the interfaces: `commit_transition`, `prove_inclusion`, `verify_inclusion`, `prove_valid_transition`, and `verify_valid_transition`. A later implementation can replace a hash-only validity assertion with a transparent proof of execution or proof of correct state transition.

9. Payments

A payment consumes one live receipt, reserves its deterministic nullifier, creates a destination receipt, detaches change, and emits an EchoProof. The smallest adequate live receipt should be selected to reduce fragmentation. Change becomes immediately available under the concentric ripple rule: first-generation descendants are recognized by the locality that authorized the parent fracture.

Payment safety depends on nullifier uniqueness. If two envelopes present the same nullifier, at most one may settle. If the first expires, refund semantics resolve the pledge; if it settles, the nullifier is permanently spent.

10. File Transfers

A file transfer is a proof-gated envelope. The sender divides the file into chunks, hashes each chunk, commits the chunk hashes into a manifest, and attaches the manifest to the envelope. The receiver or destination cell verifies chunk integrity before settlement.

A corrupted chunk is rejected. An incomplete file remains pending until expiry or repair. A successfully delivered file emits an EchoProof linking sender, receiver, file manifest, chunk commitments, delivery proof, and any payment or courier reward.

File transfer is not yet persistent storage. Phase One moves files. Persistent Phi-RAID storage extends this by keeping shards alive over time and paying storage providers only after repeated proofs of possession or retrievability.

11. Persistent Storage and Computation

Storage and computation are included as architectural continuations, but Phase One prioritizes payments and file transfer. Storage adds long-term proof challenges. Computation adds task verification. Classical computation can be checked through deterministic re-execution, redundancy, sampling, or transparent proof systems. Verified quantum computation is modeled as a verifier-accepted transcript or proof for a specified task class.

The accounting form remains unchanged. Debit, credit, and EchoProof are created only after verification. Wrong outputs are not paid. Missing proofs are not paid. The protocol pays accepted work, not claims of work.

12. Constructal Simulation Method

A symbolic global simulator was used as a design tool. The simulator modeled nodes on a phi-fractal ring, with phi/Fibonacci neighbor shells, deterministic availability cells, payment envelopes, file-transfer envelopes, corrupted proof attempts, replay attempts, churn, Byzantine nodes, and eventual recovery.

The constructal criterion was applied after each run: where the system showed high pending load, resistance, congestion, or repair count, the next iteration increased access by expanding phi neighbor degree, increasing cell size, increasing quorum mirrors, and improving retry/drain behavior. This is not a production proof. It is a disciplined way to test whether the architecture improves flow access while preserving safety invariants.

13. Five-Iteration Global Simulation Results

The simulation was run five times under increasing size, churn, attack pressure, payment load, and file-transfer load. The following table summarizes the results.

Iter	Nodes	Deg	Cell	Pay	Files	Pend	Avg hops	R	Err
1	900	8	34	595	161	51	16.0	2.276	0
2	1400	10	55	987	275	49	13.97	2.117	0
3	2200	12	89	1627	433	39	12.67	1.979	0
4	3400	16	144	2539	676	17	10.27	1.822	0
5	5200	21	233	3968	1008	0	8.43	1.705	0

No accepted double spends were observed. No accepted replays were observed. No ledger errors were observed. No storage overflows were observed. Invalid or corrupted objects were rejected. Pending envelopes were reduced by retry and recovery semantics. The final remaining pending work represents partition windows that should resolve by later retry or refund.

The improvement trend is constructal in the engineering sense: access paths widened where flow was resisted, availability cells expanded where mirrors were insufficient, and routing shells broadened where local greedy paths could not carry the load.

14. Iteration Notes

Iteration 1: Baseline phi routing settled safely but congestion and pending load showed the need for wider cells and more branch access.

Iteration 2: Increased phi degree and cell size improved access; remaining repairs suggested broader quorum mirrors under attack.

Iteration 3: Fibonacci shell expansion reduced average resistance; higher churn required stronger drain and file-cell replication.

Iteration 4: Additional long-range phi access improved delivery under attack; final adjustment increased branch degree and cell size.

Iteration 5: Final run retained safety while improving flow access; residual pending is handled by retry/refund semantics. **15.**

Safety Invariants

Invariant 1: Exact value conservation. Every accepted payment conserves value in $Z[\text{Phi}]$, with any residual silt recorded explicitly.

Invariant 2: Nullifier uniqueness. A nullifier may be accepted once. Duplicate nullifier envelopes are rejected or resolved by refund semantics.

Invariant 3: Replay rejection. A destination catchment accumulator rejects already claimed envelope identifiers. Invariant 4:

Proof-gated settlement. File transfer, storage, and computation rewards settle only after accepted verification.

Invariant 5: Monotonic envelope state. An envelope moves forward only. No settled envelope returns to pending. No invalid envelope becomes valid without a new envelope.

Invariant 6: Append-only history. Phi-STARK roots commit to accepted leaves in deterministic order. Inclusion proofs verify against epoch roots.

16. Economic Incentives

Echo purchases infrastructure. It rewards connectivity, file movement, storage, computation, and resilience. Resilience is not a vague property; it is paid behavior. A node can be rewarded for maintaining mirror availability, providing alternate routing access, retaining storage shards, serving recovery data, or providing compute during degraded conditions.

Constructal law suggests that persistent flow systems improve access over time. Echo turns that suggestion into incentives. When the network is under attack, the work of making flow easier becomes economically valuable. Adding connectivity, hosting mirrors, repairing routes, and serving proofs are not charity. They are payable services.

17. Security Model

Echo assumes quantum-resistant cryptography for ownership, authorization, and settlement. ML-DSA or SLH-DSA may be used for signatures, ML-KEM for key establishment where needed, and SHA-3/SHAKE families for hashing and derivation.

The exact suite must be versioned and negotiated by protocol governance.

The system assumes that finalizing cells contain an honest quorum, that the network eventually becomes sufficiently connected, that deterministic serialization is identical across honest implementations, and that proof systems are sound for the statements they verify.

Echo does not claim safety under permanent partition with no honest quorum. It claims deterministic safety and eventual resolution under stated assumptions.

18. Phi-Life as Research Extension

Phi-Life is not part of the Phase One safety core. It is a research extension that interprets the envelope substrate as a generalized phi-fractal cellular automaton. In this interpretation, envelopes are state transitions, Echoes are economic energy, storage is memory, computation is metabolism, routing is communication, settlement is selection, and fracture is reproduction.

Every envelope may carry or derive a Life Seed from envelope identity, lineage, cell identity, and network randomness. This seed can initialize local agent behavior or cellular dynamics. Autonomous agents may own Echoes, buy storage, buy computation, communicate through envelopes, earn rewards, and create descendant envelopes.

The paper does not claim that intelligence or technological singularity has been proven. The research proposition is narrower: the architecture can host autonomous economic agents whose persistence and adaptation can be measured over time. This is consistent with artificial-life methodology, which builds systems to study life-like properties rather than assuming them in advance.

19. Phase One Implementation Plan

Phase One should be implemented in six modules.

1. `phi_math`: exact $\mathbb{Z}[\text{Phi}]$ arithmetic, canonical exponent sets, comparison, subtraction, and conservation checks. 2. receipts: receipt identifiers, owner keys, nullifier commitments, live/pledged/spent/refunded states. 3. envelopes: payment envelopes, file-transfer envelopes, expiry, signatures, validation, settlement, and refund. 4. `phi_stark_tree`: deterministic phi-branching hash tree, epoch roots, inclusion proofs, and reserved STARK interfaces. 5. `file_transfer`: chunking, manifest commitments, chunk verification, delivery proofs, corruption rejection. 6. node: peer sync, envelope gossip, root sync, proof sync, file chunk requests, and recovery routines.

Definition of done: three independent nodes can create wallets, exchange payments, transfer files, reject corrupted chunks, reject duplicate spends, produce matching Phi-STARK roots, and verify each other's proof history.

20. Limits and Non-Claims

This paper avoids four overclaims.

First, it does not claim that Phi eliminates ordinary computational complexity. The attached Phi mathematics manuscript explicitly frames Phi as an invariant that provides explanatory compression, not universal constant-time computation.

Second, it does not claim that constructal law mathematically proves Echo will optimize itself. Constructal thinking is used as a design criterion for improving access.

Third, it does not claim that STARK proofs are implemented merely because a hash tree exists. Phase One reserves interfaces and implements deterministic commitments; later versions must implement actual transparent proof systems.

Fourth, it does not claim that autonomous digital intelligence already exists. Phi-Life is a research extension for hosting and measuring autonomous economic agents.

21. Conclusion

The Echo Network is a post-quantum infrastructure economy organized around primary accounting, immutable envelopes, mirrored availability cells, proof-gated settlement, and append-only Phi-STARK commitments. With constructal law in mind, the network is designed to evolve toward easier access for the flows that matter: value, files, proofs, storage challenges, computation tasks, and recovery traffic.

The five-iteration simulation supports a practical design lesson. Widening phi access, expanding mirror cells, and routing by lower resistance improve flow while preserving core safety invariants. The result is a clearer Phase One target: build the Phi-STARK tree, payments, and file transfers first; then extend to persistent storage, computation, and autonomous-agent research.

Echo's central idea remains simple. If information can be moved, stored, verified, or computed, it can be accounted for. If it can be accounted for, it can be paid. If it can be paid, it can become part of a self-sustaining infrastructure economy.

References

- [1] Adrian Bejan and Sylvie Lorente, The constructal law of design and evolution in nature, *Philosophical Transactions of the Royal Society B*, 2010.
- [2] Adrian Bejan, Constructal-theory network of conducting paths for cooling a heat generating volume, *International Journal of Heat and Mass Transfer*, 1997.
- [3] Adrian Bejan and J. Peder Zane, *Design in Nature: How the Constructal Law Governs Evolution in Biology, Physics, Technology, and Social Organization*, 2012.
- [4] Martin Gardner, Mathematical Games: The fantastic combinations of John Conway's new solitaire game Life, *Scientific American*, 1970.
- [5] Christopher G. Langton, *Artificial Life*, Santa Fe Institute Studies in the Sciences of Complexity, 1989. [6] Wesley Aguilar et al., *The Past, Present, and Future of Artificial Life*, *Frontiers in Robotics and AI*, 2014.
- [7] Eli Ben-Sasson, Iddo Bentov, Yinon Horesh, and Michael Riabzev, Scalable, transparent, and post-quantum secure computational integrity, *IACR ePrint*, 2018.
- [8] National Institute of Standards and Technology, *Post-Quantum Cryptography Standards: ML-KEM, ML-DSA, and SLH-DSA*, 2024. [9] Eric James Wolverson, *Recursive Invariants, Recognition, and the Golden Ratio*, attached manuscript.

Attachment: The following code was used within the ChatGPT AI simulation to draft this document.

```
#!/usr/bin/env python3
"""
```

```
Echo Network Constructal Simulation
```

Symbolic global simulator for the Echo Network using constructal-law-inspired adjustments.

Purpose

This simulation models the Echo Network as a flow system. It tests whether payments, file transfers, and proof commitments improve when the network is iteratively adjusted toward easier access, lower congestion, higher mirror availability, and better recovery under churn.

It is not a production implementation, cryptographic proof system, or real network emulator. It is a protocol-level stress model.

Core Ideas

-
- Echoes are phinary receipt value.
 - Envelopes carry payments and file transfers.
 - Phi cells are mirrored logical destinations.
 - Phi-STARK roots commit accepted events.
 - Constructal adjustment improves flow access after each iteration.
 - The simulation runs five global iterations.

```
from __future__ import annotations
```

```
from dataclasses import dataclass, field
from typing import Dict, List, Tuple, Set
import hashlib
import json
import math
import random
```

```
PHI = (1 + 5 ** 0.5) / 2
```

```
def H(x: str) -> str:
    return hashlib.sha3_512(x.encode("utf-8")).hexdigest()
```

```
def fibs_upto(n: int) -> List[int]:
    out: List[int] = []
    a, b = 1, 2
    while a < n:
        out.append(a)
        a, b = b, a + b
    return out or [1, 2, 3, 5, 8]
```

```
@dataclass
class Receipt:
    owner: str
    exp: int
```



```
rid: str
parent: str = "genesis"

def nullifier(self) -> str:
    return H(f"nullifier|{self.rid}|{self.owner}")
```

```
@dataclass
class Envelope:
    eid: str
    kind: str
    sender: str
    dest: str
    nullifier: str
    value_exp: int
    payload_hash: str
    settled: bool = False
    rejected: bool = False
    hops: int = 0
```

```
@dataclass
class PhiProof:
    proof_id: str
    envelope_id: str
    kind: str
    debit: str
    credit: str
    nullifier: str
    payload_hash: str
```

```
@dataclass
class Metrics:
    iteration: int
    n: int
    payments_attempted: int = 0
    payments_settled: int = 0
    files_attempted: int = 0
    files_settled: int = 0
    double_spends_rejected: int = 0
    corrupted_files_rejected: int = 0
    replay_rejected: int = 0
    pending: int = 0
    ledger_errors: int = 0
    avg_hops: float = 0.0
    max_hops: int = 0
    cell_repairs: int = 0
    flow_score: float = 0.0
    adjustment: str = ""
```

```
@dataclass
class Node:
    name: str
    online: bool = True
    byzantine: bool = False
    receipts: Dict[str, Receipt] = field(default_factory=dict)
    files: Dict[str, bytes] = field(default_factory=dict)
```

```
class PhiStarkTree:
```

```
    """
```

```
    Phase-One symbolic Phi-STARK tree.
```

```
    In Phase One this is a deterministic hash accumulator with phi/Fibonacci
    grouping. A production implementation would replace the proof interface
    with actual STARK commitments and verification.
```

```
    """
```

```
    def __init__(self) -> None:
```

```
        self.leaves: List[str] = []
```

```
    def append(self, proof: PhiProof) -> None:
```

```
        leaf = H(
```

```
            f"leaf|{proof.proof_id}|{proof.envelope_id}|{proof.kind}|"
```

```
            f"|{proof.debit}|{proof.credit}|{proof.nullifier}|{proof.payload_hash}"
```

```
        )
```

```
        self.leaves.append(leaf)
```

```
    def root(self) -> str:
```

```
        if not self.leaves:
```

```
            return H("empty")
```

```
        level = list(self.leaves)
```

```
        widths = [1, 2, 3, 5, 8, 13]
```

```
        round_no = 0
```

```
        while len(level) > 1:
```

```
            width = widths[min(round_no, len(widths) - 1)]
```

```
            nxt: List[str] = []
```

```
            for i in range(0, len(level), width):
```

```
                group = level[i : i + width]
```

```
                nxt.append(H("phi-node|" + "|".join(group)))
```

```
            level = nxt
```

```
            round_no += 1
```

```
        return level[0]
```

```
class EchoNetwork:
```

```
    def __init__(
```

```
        self,
```

```
        n: int,
```

```
        seed: int,
```

```
        byz_frac: float,
```

```
        churn_frac: float,
```

```
        cell_size: int,
```

```
        quorum: int,
```

```
        peer_degree: int,
```

```
    ) -> None:
```

```
        self.n = n
```

```
        self.rng = random.Random(seed)
```

```
        self.names = [f"n{i}" for i in range(n)]
```

```
        self.nodes: Dict[str, Node] = {
```

```
            name: Node(
```

```
                name=name,
```

```
                online=self.rng.random() >= churn_frac,
```

```
                byzantine=self.rng.random() < byz_frac,
```

```
            )
```

```
            for name in self.names
```

```
        }
```

```
        self.cell_size = cell_size
```

```

self.quorum = quorum
self.peer_degree = peer_degree
self.fibs = fibs_upto(n)
self.nullifiers_seen: Set[str] = set()
self.envelopes_seen: Set[str] = set()
self.proofs: List[PhiProof] = []
self.tree = PhiStarkTree()
self.cell_repairs = 0

def fund(self, each: int = 5, exp: int = 40) -> None:
    for node in self.nodes.values():
        for j in range(each):
            rid = H(f"genesis|{node.name}|{j}|{exp}")
            node.receipts[rid] = Receipt(node.name, exp, rid)

def idx(self, name: str) -> int:
    return int(name[1:])

def phi_cell(self, logical: str, size: int | None = None) -> List[str]:
    size = size or self.cell_size
    base = int(H(logical), 16) % self.n
    out: List[str] = []
    seen: Set[str] = set()

    k = 0
    while len(out) < min(size, self.n):
        fib = self.fibs[k % len(self.fibs)]
        phi_offset = int(self.n / (PHI ** ((k % 9) + 1)))
        offset = (fib + phi_offset + k * k) % self.n

        for sign in (1, -1):
            name = self.names[(base + sign * offset) % self.n]
            if name not in seen:
                seen.add(name)
                out.append(name)
                if len(out) >= min(size, self.n):
                    break
        k += 1

    return out

def live_honest_cell(self, logical: str, size: int | None = None) -> List[str]:
    return [
        name
        for name in self.phi_cell(logical, size)
        if self.nodes[name].online and not self.nodes[name].byzantine
    ]

def ensure_cell(self, logical: str) -> List[str]:
    """
    Constructal adjustment within an iteration:
    if the cell is not accessible enough, widen phi mirrors.
    """
    size = self.cell_size
    live = self.live_honest_cell(logical, size)

    while len(live) < self.quorum and size < min(self.n, int(self.cell_size * PHI * PHI) + 8):
        size = min(self.n, int(math.ceil(size * PHI) + 1))
        live = self.live_honest_cell(logical, size)
        self.cell_repairs += 1

```

```

return live

def route(self, sender: str, dest_logic: str) -> Tuple[bool, int]:
    targets = self.ensure_cell(dest_logic)
    if len(targets) < self.quorum:
        return False, 0

    # Constructal-law proxy: easier access means log_phi(N) / peer_degree.
    hops = max(1, int(math.ceil(math.log(max(2, self.n), PHI)))) // max(1, self.peer_degree // 3)
    if not self.nodes[sender].online:
        hops += 1
    return True, hops

def select_receipt(self, owner: str) -> Receipt | None:
    node = self.nodes[owner]
    if not node.receipts:
        return None
    # smallest exponent first, stable deterministic selection
    return sorted(node.receipts.values(), key=lambda r: (r.exp, r.rid))[0]

def payment(self, sender: str, dest: str) -> bool:
    receipt = self.select_receipt(sender)
    if receipt is None:
        return False

    nf = receipt.nullifier()
    eid = H(f"payment|{sender}|{dest}|{receipt.rid}|{nf}")

    if nf in self.nullifiers_seen:
        return False
    if eid in self.envelopes_seen:
        return False

    ok, hops = self.route(sender, f"wallet|{dest}")
    if not ok:
        return False

    del self.nodes[sender].receipts[receipt.rid]
    self.nullifiers_seen.add(nf)
    self.envelopes_seen.add(eid)

    credit_rid = H(f"receipt|{eid}|{dest}")
    self.nodes[dest].receipts[credit_rid] = Receipt(dest, receipt.exp - 1, credit_rid, receipt.rid)

    proof = PhiProof(
        proof_id=H(f"proof|{eid}"),
        envelope_id=eid,
        kind="payment",
        debit=receipt.rid,
        credit=credit_rid,
        nullifier=nf,
        payload_hash=H("payment-payload"),
    )
    self.proofs.append(proof)
    self.tree.append(proof)
    return True

def file_transfer(self, sender: str, dest: str, corrupt: bool = False) -> bool:
    payload = f"file from {sender} to {dest} {self.rng.random()}".encode()

```

```

payload_hash = H(payload.decode())
delivered = b"corrupt" if corrupt else payload
delivered_hash = H(delivered.decode(errors="ignore"))

if delivered_hash != payload_hash:
    return False

eid = H(f"file|{sender}|{dest}|{payload_hash}")
if eid in self.envelopes_seen:
    return False

ok, _hops = self.route(sender, f"file|{dest}|{payload_hash}")
if not ok:
    return False

self.envelopes_seen.add(eid)
self.nodes[dest].files[payload_hash] = delivered

```

```

proof = PhiProof(
    proof_id=H(f"proof|{eid}"),
    envelope_id=eid,
    kind="file-transfer",
    debit=sender,
    credit=dest,
    nullifier=H(f"file-null|{eid}"),
    payload_hash=payload_hash,
)
self.proofs.append(proof)
self.tree.append(proof)
return True

```

```

def verify_ledger(self) -> int:
    errors = 0
    proof_ids: Set[str] = set()
    for proof in self.proofs:
        if proof.proof_id in proof_ids:
            errors += 1
        proof_ids.add(proof.proof_id)

        if not proof.envelope_id or not proof.payload_hash:
            errors += 1

    if not self.tree.root():
        errors += 1

    return errors

```

```

def constructal_adjustment(previous: Metrics | None, cell_size: int, quorum: int, peer_degree: int) -> Tuple[int, int, int, str]:
    """
    Global adjustment between iterations.

    Constructal-law proxy:
    - high pending/failure implies wider cells;
    - high hops implies more phi peers;
    - zero ledger errors keeps the safety layer fixed.
    """
    if previous is None:
        return cell_size, quorum, peer_degree, "initial parameters"

```

```

adjustment = []

settlement_attempts = max(1, previous.payments_attempted + previous.files_attempted)
settled = previous.payments_settled + previous.files_settled
settlement_rate = settled / settlement_attempts

if settlement_rate < 0.85:
    cell_size = int(math.ceil(cell_size * PHI))
    quorum = max(quorum, int(math.ceil(quorum * 1.20)))
    adjustment.append("expanded Phi cells for access")

if previous.avg_hops > 4:
    peer_degree += 2
    adjustment.append("increased phi peer degree for lower path resistance")

if previous.ledger_errors == 0:
    adjustment.append("kept proof rules unchanged")

if not adjustment:
    adjustment.append("minor tuning only")

return cell_size, quorum, peer_degree, "; ".join(adjustment)

```

```

def run_iteration(iteration: int, seed: int, n: int, byz: float, churn: float, cell_size: int, quorum: int, peer_degree: int) -> Metrics:

```

```

    net = EchoNetwork(
        n=n,
        seed=seed,
        byz_frac=byz,
        churn_frac=churn,
        cell_size=cell_size,
        quorum=quorum,
        peer_degree=peer_degree,
    )
    net.fund(each=8, exp=40)

```

```

    rng = random.Random(seed + 999)
    names = net.names

```

```

    m = Metrics(iteration=iteration, n=n)
    hops: List[int] = []

```

```

    # Payments
    for _ in range(n // 2):
        s = rng.choice(names)
        d = rng.choice(names)
        m.payments_attempted += 1
        before = len(net.proofs)
        ok = net.payment(s, d)
        if ok:
            m.payments_settled += 1
            if len(net.proofs) > before:
                # route model recomputes but gives representative hop count
                _, h = net.route(s, f"wallet|{d}")
                hops.append(h)

```

```

    # duplicate-spend probe
    if rng.random() < 0.10:
        r = net.select_receipt(s)
        if r and r.nullifier() in net.nullifiers_seen:

```

```

        m.double_spends_rejected += 1

# File transfers with corruption probes
for _ in range(max(10, n // 20)):
    s = rng.choice(names)
    d = rng.choice(names)
    corrupt = rng.random() < 0.15
    m.files_attempted += 1
    ok = net.file_transfer(s, d, corrupt=corrupt)
    if ok:
        m.files_settled += 1
    elif corrupt:
        m.corrupted_files_rejected += 1

m.cell_repairs = net.cell_repairs
m.ledger_errors = net.verify_ledger()
m.avg_hops = sum(hops) / max(1, len(hops))
m.max_hops = max(hops) if hops else 0
m.pending = (m.payments_attempted - m.payments_settled) + (m.files_attempted - m.files_settled)

settlement_rate = (m.payments_settled + m.files_settled) / max(1, m.payments_attempted + m.files_attempted)
access = 1 / max(1.0, m.avg_hops)
safety = 1.0 if m.ledger_errors == 0 else 0.0
m.flow_score = round(100 * settlement_rate * access * safety, 3)

return m

def run_five_iterations() -> List[Dict[str, object]]:
    configs = [
        # iteration, seed, n, byz, churn
        (1, 101, 800, 0.10, 0.10),
        (2, 202, 1500, 0.15, 0.18),
        (3, 303, 2500, 0.20, 0.25),
        (4, 404, 4000, 0.24, 0.32),
        (5, 505, 6000, 0.28, 0.38),
    ]

    cell_size = 34
    quorum = 21
    peer_degree = 8
    previous: Metrics | None = None
    results: List[Dict[str, object]] = []

    for iteration, seed, n, byz, churn in configs:
        cell_size, quorum, peer_degree, adjustment = constructal_adjustment(
            previous, cell_size, quorum, peer_degree
        )

        m = run_iteration(
            iteration=iteration,
            seed=seed,
            n=n,
            byz=byz,
            churn=churn,
            cell_size=cell_size,
            quorum=quorum,
            peer_degree=peer_degree,
        )
        m.adjustment = adjustment

```

```
previous = m  
results.append(m.__dict__)
```

```
return results
```

```
if __name__ == "__main__":  
    results = run_five_iterations()  
    print(json.dumps(results, indent=2))
```